

APP DEV PROJECT REPORT

1. Student Details

- **Name:** Sagar Gupta
- **Roll Number:** 21F1000539
- **Email:** 21f1000539@ds.study.iitm.ac.in
- **About Me:** I have done my bachelor's in mathematics (B.Sc. in Mathematics) along with this BS degree, and I have worked as Business Head in Avishkar hyperloop.

2. Project Details

- **Project Title:** Hospital Management System - V2
- **Problem Statement:** To design and build a comprehensive role-based web application that streamlines hospital operations. It allows admins to manage doctors and appointments, doctors to manage their availability and provide treatment details, and patients to register, book appointments, check their medical history, and export data.
- **Approach:** The app follows a decoupled architecture using Flask as the backend API and Vue 3 for a reactive frontend single-page application (SPA). It uses SQLite for data persistence via SQLAlchemy. For asynchronous processes (such as scheduled reminders, monthly summaries, and data exports), Redis and Celery have been integrated to prevent UI blocking and improve performance.

3. AI/LLM Declaration

I used AI tools like ChatGPT/Gemini to assist in writing standard boilerplate code (e.g., SQLAlchemy model configurations), creating API structures, setting up background jobs (Celery configurations), and frontend layout templates.

The extent of AI/LLM usage is around 15–20%, limited to syntax suggestions and resolving configuration errors. All core logic, integration, structure decisions, debugging, and final application flow were built manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework providing RESTful APIs
SQLAlchemy	Object Relational Mapper for managing the

	SQLite database
Vue 3	Frontend JavaScript framework used for building interactive UI
Vue Router	Handling frontend routing within the Single Page Application
Axios	Making HTTP requests from the Vue frontend to the Flask backend
Bootstrap 5	Frontend CSS framework for responsive layout and styling
Celery	Handling background computing tasks like daily reminders and async CSV exports
Redis	In-memory message broker used along with Celery and for caching
SQLite	Lightweight local database for storing all user and operational data

5. Database Schema / ER Diagram

- **Tables:**
 - **User:** stores generic user login logic (id, name, email, role, password, is_active, created_at)
 - **Department:** hospital departments (id, name, description)
 - **DoctorProfile:** doctor details, linked to User and Department (id, user_id, department_id, specialization, phone_number, etc.)
 - **PatientProfile:** patient details, linked to User (id, user_id, age, gender, medical_history, address, etc.)
 - **Appointment:** links DoctorProfile and PatientProfile for bookings (id, doctor_id, patient_id, date, time, reason, status)
 - **Treatment:** medical details for a specific appointment (id, appointment_id, diagnosis, prescription, notes, next_visit_date)
 - **DoctorAvailability:** doctor's working schedule (id, doctor_id, date, start_time, end_time, is_available)
 - **ExportRequest:** tracks CSV export task status (id, patient_id, status, file_path)
- **Relationships:**
 - One-to-One → User to DoctorProfile
 - One-to-One → User to PatientProfile
 - One-to-Many → Department to DoctorProfile
 - One-to-Many → DoctorProfile to Appointment

- One-to-Many → PatientProfile to Appointment
- One-to-One → Appointment to Treatment
- One-to-Many → DoctorProfile to DoctorAvailability
- One-to-Many → PatientProfile to ExportRequest

(Insert ER diagram image here – can be created using dbdiagram.io or draw.io)

6. API Resource Endpoints

(Subset of core APIs available)

Endpoint	Method	Description
/api/auth/login	POST ▾	Authenticate user and generate session/token
/api/auth/register	POST ▾	Register a new patient account
/api/admin/dashboard	GET ▾	Fetch dashboard statistics for the hospital admin
/api/doctor/appointments	GET ▾	List appointments for logged-in doctor
/api/doctor/availability	POST ▾	Update the doctor's weekly available slots
/api/patient/appointments	POST ▾	Book a new appointment with a doctor
/api/patient/export	POST ▾	Trigger background job to export patient's treatment history

7. Architecture and Features (optional)

● Architecture Overview:

- app.py – main Flask application entry point with route blueprint registrations
- models.py – SQLAlchemy schema bindings
- celery_app.py – Celery configuration and background worker initialization
- /routes – Flask Blueprints modularizing the API based on entity (auth, admin,

doctor, patient)

- `frontend/src/views` – Vue pages for the Single Page Application
- `frontend/src/components` – Reusable UI pieces

- **Implemented Features:**

- Role-based login system (Admin, Doctor, Patient)
- Doctor scheduling and appointment conflict management
- Patient medical history tracking and treatment updating
- Scheduled tasks: Daily generic reminders and Monthly activity reports
- Asynchronous CSV data export using Celery and Redis
- RESTful JSON API layer securely communicating with modern Vue3 UI
- Clean and responsive view constructed with Bootstrap 5

8. Video Presentation

- **Drive Link:**

<https://drive.google.com/drive/folders/1qKyToZ8v8XnylP92Xjx8Oz9Lmq4W9pw6?usp=sharing>

(Accessible to all with “View” permission.)